7zpus.swe@gmail.com

# Specifica Tecnica

## Tabella di Versionamento

| Versione | Data | Autore | Verificatore | Descrizione |
|---|---|---|---|---|
| 0.1.0 | 2026/02/28 | Laoud Zakaria | Georgescu Diana | Creazione del documento e stesura struttura iniziale |

# Indice

## List of Figures

## List of Tables

# 1 Introduzione

## 1.1 Obiettivo del documento

L'obiettivo del presente documento è descrivere in modo rigoroso le scelte tecnologiche e l'architettura del prodotto *DIPReader*, adottando il modello di progettazione *C4*.
L'analisi verrà condotta partendo dalla descrizione del contesto applicativo, per poi approfondire la struttura dei container, delle componenti e, infine, del codice, al fine di fornire una rappresentazione completa, coerente e sistematica del sistema.

## 1.2 Glossario

Ogni termine tecnico o con un significato particolare nell'ambito dell'Ingegneria del Software, utilizzato nella documentazione di progetto, è definito nell'apposito documento Glossario (v2.0.0).

## 1.3 Riferimenti

### 1.3.1 Riferimenti normativi

- Norme di Progetto v2.0.0 (v2.0.0)
- Standard SDD/IEEE/1016:2009(ultimo accesso: 02/03/2026)
- Regolamento di Progetto Didattico a.a. 2025/2026 (ultimo accesso: 02/03/2026)

### 1.3.2 Riferimenti informativi

- Capitolato C3: DIPReader (ultimo accesso: 10/02/2026)
- Dispense su Dependency Injection (Ultimo accesso: 02/03/2026)
- Dispense su OOP (Ultimo accesso: 02/03/2026)
- Dispense su Diagrammi delle classi (Ultimo accesso: 02/03/2026)
- Dispense su Pattern architetturali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern creazionali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern strutturali (Ultimo accesso: 02/03/2026)
- Dispense su Principi SOLID (Ultimo accesso: 02/03/2026)
- Dispense sulla Progettazione software (Ultimo accesso: 02/03/2026)
- Dispense sulla Qualità del software (Ultimo accesso: 02/03/2026)
- Repository Architettura esagonale (Kotlin) (Ultimo accesso: 02/03/2026)

- Repository Architettura esagonale (Java) (Ultimo accesso: 02/03/2026)
- Repository Ingegneria del software - swe-imdb (Ultimo accesso: 02/03/2026)
- C4 Model (Ultimo accesso: 03/03/2026)
- PlantUML (Ultimo accesso: 03/03/2026)
- Riferimenti a scelte tecnologiche:
  - TypeScript (Ultimo accesso: 03/03/2026)
  - Angular (Ultimo accesso: 03/03/2026)
  - Electron (Ultimo accesso: 03/03/2026)
  - SQLite (Ultimo accesso: 03/03/2026)
  - sqlite-vss (Ultimo accesso: 03/03/2026)
  - ONNX Runtime (Ultimo accesso: 03/03/2026)
  - Transformers.js (Ultimo accesso: 03/03/2026)
  - Node.js (Ultimo accesso: 03/03/2026)
- Glossario (v2.0.0)
- Piano di Progetto (v2.0.0)
- Piano di Qualifica (v2.0.0)
- Verbali interni:
- Verbali esterni:

# 2 Tecnologie utilizzate

La scelta delle teccnologie è stata guidata dalla necessità di rispetta i vincoli richiesti dalla proponente legati ad autoconsistenza, portabilità e facilità d'uso, nonché da considerazioni relative alla facilità di sviluppo, alla disponibilità di risorse e alla compatibilità con i requisiti funzionali e non funzionali del sistema.

# 3 Linguaggio di Programmazione: TypeScript

| Scelta | Versione | Motivazione |
|---|---|---|
| TypeScript | 5.9.2 | Scelto per la sua capacità di fornire un sistema di tipi statico che migliora la sicurezza del codice e facilita la manutenzione, riducendo il rischio di errori comuni durante lo sviluppo. Inoltre, TypeScript è ampiamente supportato da un ecosistema ricco di librerie e strumenti, e si integra perfettamente con tecnologie come Angular ed Electron. |

Table 2: Dettagli sulla scelta di TypeScript come linguaggio di programmazione

## 3.1 Deployment: Electron

| Scelta | Versione | Motivazione |
|---|---|---|
| Electron | 30.0.0 | Scelto per la sua capacità di consentire lo sviluppo di applicazioni desktop multipiattaforma utilizzando tecnologie web, garantendo così la portabilità del prodotto su diversi sistemi operativi. Electron permette di creare un'esperienza utente nativa, sfruttando le potenzialità di Node.js per l'accesso al filesystem e alle risorse locali, facilitando l'integrazione con librerie e strumenti di terze parti. |

Table 4: Dettagli sulla scelta di Electron come piattaforma di deployment

## 3.2 Framework Frontend: Angular

| Scelta | Versione | Motivazione |
|---|---|---|
| Angular | 21.2.5 | Scelto per la sua capacità di supportare lo sviluppo di applicazioni web complesse e scalabili, con un'architettura basata su componenti che favorisce la modularità e la manutenibilità del codice. Inoltre, Angular offre un'ampia gamma di strumenti e funzionalità integrate, come la gestione dello stato, il routing e il supporto per la creazione di interfacce utente reattive, che lo rendono adatto a soddisfare i requisiti funzionali del progetto. La sua implementazione della Dependency Injection facilita la gestione delle dipendenze tra i componenti, migliorando la testabilità e la manutenibilità del codice. |

Table 6: Dettagli sulla scelta di Angular come framework frontend

## 3.3 Librerie Aggiuntive

| Scelta | Versione | Motivazione |
|---|---|---|
| Better SQLite | 12.8.0 | SQLite è un database relazionale che non richiede un server separato e può essere integrato direttamente nell'applicazione, facilitando lo sviluppo e il deployment. La sua leggerezza e portabilità lo rendono adatto a un'ampia gamma di piattaforme, garantendo la compatibilità con i requisiti di portabilità del progetto. |
| sqlite-vss | 0.1.2 | Scelto per la sua capacità di fornire una versione virtuale di SQLite, che consente di eseguire query su dati in memoria senza la necessità di creare un database fisico dei vettori per la ricerca semantica, semplificando così l'integrazione con il sistema. |
| ONNX Runtime | 1.17.3 | Scelto per la sua capacità di eseguire modelli di machine learning in modo efficiente e flessibile, supportando una vasta gamma di modelli e framework, facilitando l'integrazione di funzionalità di intelligenza artificiale nel prodotto. |
| Transformers.js | 3.8.1 | Scelto per la sua capacità di fornire un'interfaccia JavaScript per l'utilizzo di modelli di trasformatori, facilitando l'integrazione di funzionalità avanzate di elaborazione del linguaggio naturale e machine learning nel prodotto. |

Table 8: Dettagli sulle scelte di librerie aggiuntive

## 3.4 Tecnologie di analisi statica

| Scelta | Versione | Motivazione |
|---|---|---|
| SonarQube | 2026.2 | Scelto per la sua capacità di identificare e segnalare pattern di codice problematici, migliorando la qualità del codice e facilitando la manutenzione del progetto. Integrato nella pipeline di sviluppo, SonarQube consente di monitorare costantemente la qualità del codice, identificando potenziali bug, vulnerabilità e problemi di manutenzione, contribuendo così a garantire un prodotto finale robusto e affidabile. |
| Prettier | 2.9.3 | Scelto per la sua capacità di formattare automaticamente il codice in modo coerente, migliorando la leggibilità e facilitando la collaborazione tra i membri del team. Prettier supporta una vasta gamma di linguaggi e può essere facilmente integrato con editor di codice e strumenti di sviluppo, semplificando il processo di mantenimento di uno stile di codifica uniforme. |

Table 10: Dettagli sulle scelte di tecnologie di analisi statica

## 3.5 Tecnologie di analisi dinamica

| Scelta | Versione | Motivazione |
|---|---|---|
| Vitest | 2.3.1 | Scelto per la sua capacità di fornire un ambiente di test per applicazioni JavaScript e per il suo controllo dei componenti di Mock. |
| Playwright | 1.59.1 | Scelto per la sua capacità di fornire un ambiente di test per applicazioni web, con possibilità di testare flussi completi di interazione utente. Utilizzato quindi per i test E2E. |

Table 12: Dettagli sulle scelte di tecnologie di analisi dinamica

# 4 Contesto del sistema

I vincoli di progetto, come specificato nell'Analisi dei Requisiti, richiedono che il sistema sia portabile, facile da usare e autoconsistente. Il caso d'uso principale del sistema è quello di un utente che desidera esplorare e interrogare un DIP per estrarre documenti fiscali e non, potendone verificare la validità e autenticità.

## 4.1 Modello architetturale

I vincoli di progetto hanno guidato la progettazione del sistema, influenzando le scelte tecnologiche e architetturali. Il sistema progettato quindi è pensato per non utilizzare tecnologie che richiedano installazioni o configurazioni complesse da parte dell'utente. I servizi offerti sono tutti accessibili in assenza di connettività, complesa la ricerca semantica.
Gli unici attori esterni coinvolti sono il file *system locale* da cui viene letto il DIP. STAMPA DA VEDERE SE HA SENSO METTERLO

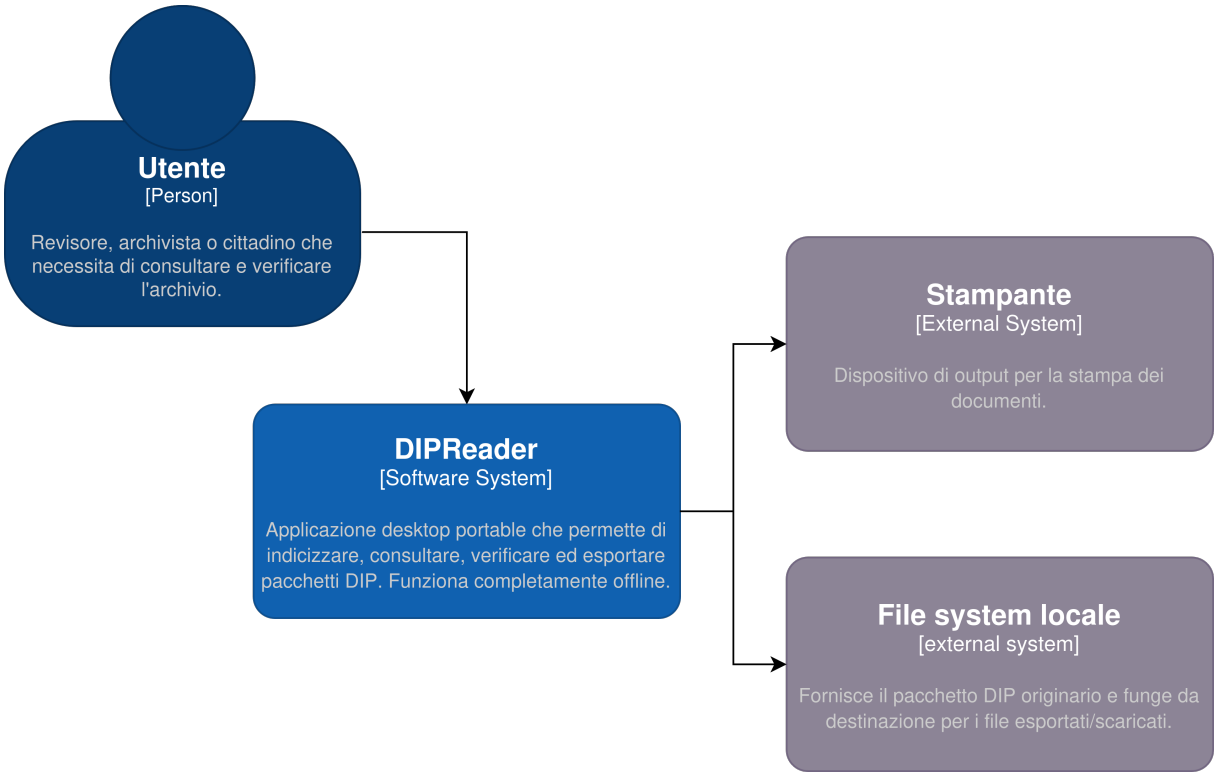


Figure 1: Diagramma C1 - Contesto del sistema *DIPReader*

## 4.2 Descrizione degli attori esterni

Gli unici attori esterni come già detto sono:

- il file system locale da cui viene letto il DIP
- STAMPA DA VEDERE SE HA SENSO METTERLO

# 5 Architettura dei container

La progettazione del sistema *DIPReader* è stata guidata dai principi di modularità, scalabilità e manutenibilità, con l'obiettivo di creare un'architettura che possa evolversi nel tempo e adattarsi a nuove esigenze e normative AGID.
In particolare, è stata completamente scorporata la logica di business dalla logica di presentazione, esponendo come unico punto di accesso comune gli *IPC channel*.
Il sistema così facendo permette di avere un backend facilmente estendibile, modificabile e soprattutto che si adatta allla struttura del DIP autonomamente. D'altra parte il frontend è completamente disaccoppiato ma strutturato in modo che i componenti di presentazione siano facilmente sostituibili, modificabili e soprattutto adattabili a nuove esigenze.

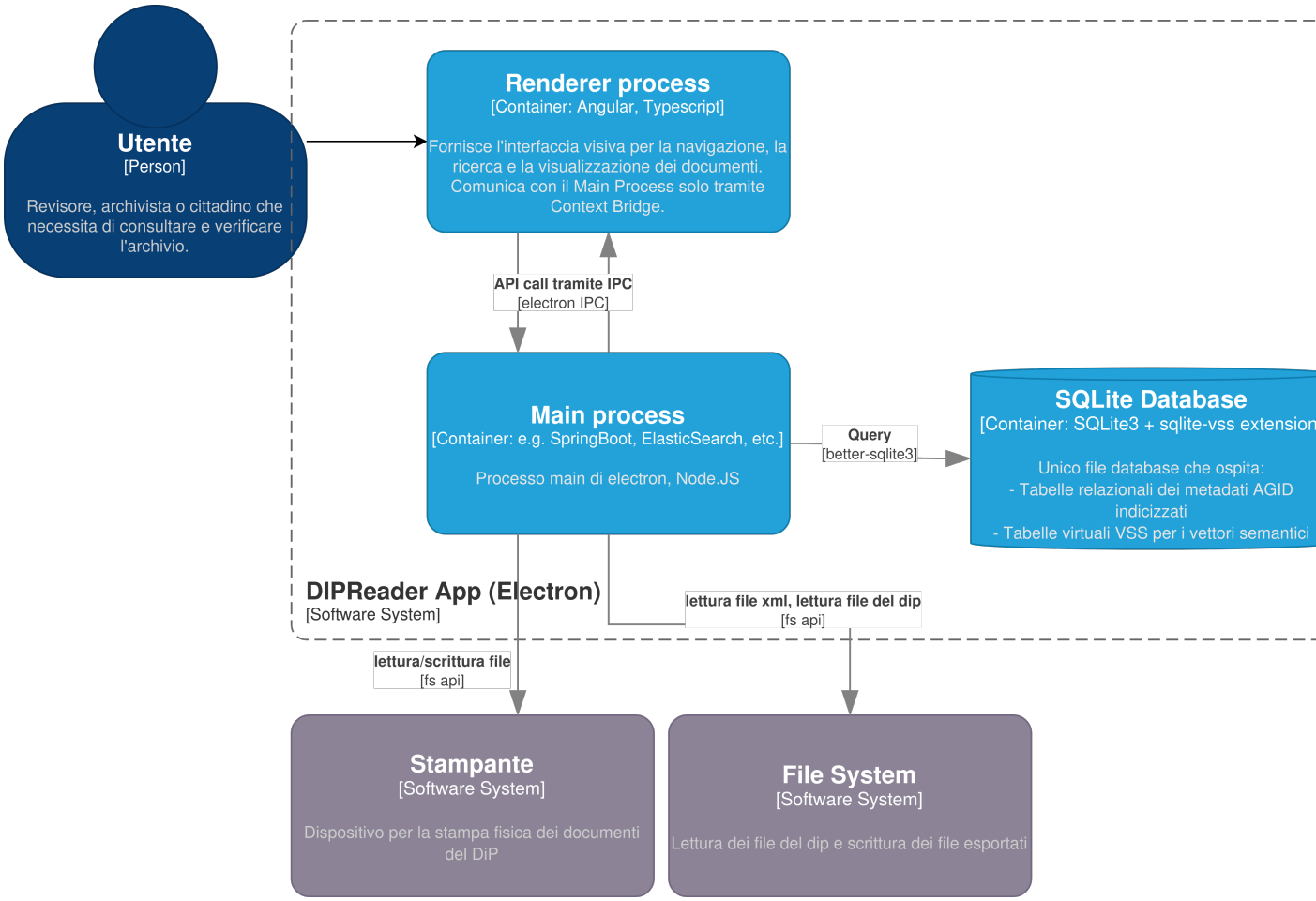


Figure 2: Diagramma C2 - Container del sistema *DIPReader*

## 5.1 Renderer

### 5.1.1 Tecnologia: Angular

#### Descrizione

## 5.2 [Main]

### 5.2.1 Tecnologia: Node.js

#### Descrizione

# 6 Architettura delle componenti

## 6.1 [Componente 1]

### 6.1.1 Descrizione

### 6.1.2 Interfacce

### 6.1.3 Dipendenze

### 6.1.4 Tracciamento dei requisiti

## 6.2 [Componente 2]

### 6.2.1 Descrizione

### 6.2.2 Interfacce

### 6.2.3 Dipendenze

### 6.2.4 Tracciamento dei requisiti